Booleans and Decisions

AP CSP · Unit 2, Lesson 1 · Read page 1 before class

Every program you have written so far does the same thing every time it runs. This is the page where that stops.

How to use this sheet. Read page 1 before the lesson — the goal is that when these words come up in the period, it is the second time you have seen them, not the first. Keep it next to you while you build; the examples use your lander. **Leave page 2 face-down** until we get there.

A third kind of value BOOLEAN

Every value you have written so far has been a **number** or a piece of **text**. Here is a third kind, and it has only two possible values, ever: `true` and `false`.

You get one whenever you *compare* two things:

```
// landerY is 240
landerY > 200    // true
landerY < 200    // false
landerY >= 240   // true
```

A comparison is not a calculation. It does not hand back a number — it hands back an **answer**. A value like that is called a **Boolean**.

The four types the AP exam names

TYPE	YOU HAVE WRITTEN
number	`let landerY = 100;`
string	`"ALT " + landerY`
Boolean	`landerY > 200`
list	Unit 4

WORTH KNOWING
The College Board names these four directly (**AAP-1.A.3**). You have been using two of them since Lesson 4.

NOT THE SAME THING
`true` is a Boolean.
`"true"` — with quotes — is a **string**, just text. They look alike and behave nothing alike.

Six ways to compare two things RELATIONAL OPERATORS

SYMBOL	READ IT OUT LOUD AS	IF LANDERY IS 240
`>`	greater than	`landerY > 200 → true`
`<`	less than	`landerY < 200 → false`
`>=`	greater than **or equal to**	`landerY >= 240 → true`
`<=`	less than **or equal to**	`landerY <= 200 → false`
`==`	is the same as	`landerY == 240 → true`
`!=`	is not the same as	`landerY != 200 → true`

READ THE WHOLE SYMBOL
`>=` is "greater than *or equal to*", not "greater than". That second half is doing real work, and skimming past it is one of the easiest ways to answer a question wrong.

Give the answer a name A BOOLEAN IS A VALUE

A comparison hands back a value, and a value can be **stored** — the same way you store a number.

Naming a Boolean

The name should read back as a **yes / no question**:

```
let low = landerY > 200;   // true

if (low) {
  // ...
}
```

Nothing new is happening. It is an ordinary `let`. The only new part is *what* the value is.

> **WHY IT IS WORTH DOING**
> You can **look at it**, and you can use the same answer in more than one place without asking the question again.

GOOD	READS AS
`landed`	"landed?"
`isFalling`	"is falling?"
`hasFuel`	"has fuel?"

> **WHY THE NAME MATTERS**
> `y = landerY > 200` runs fine, but `y` could be holding anything. `low` can only be holding an *answer*.

> **ON THE EXAM**
> **AAP-2.F.5** says an operand can be a Boolean *expression* or a single Boolean *value*. That second half is this page.

The opposite of an answer `!` · NOT ON THE EXAM

```
// low is true
!low            // false
!(landerY > 200)  // false
```

`!` flips a Boolean. True becomes false, false becomes true. It is the only one of these that takes a single answer rather than two things to compare.

Three ways to say "not low"

WRITTEN	ALWAYS RIGHT?
`!low`	yes
`low == false`	yes
`landerY <= 200`	only if you flip it correctly

> **THE THIRD ONE IS WHERE PEOPLE SLIP**
> The opposite of `>` is `<=`, not `<`. Get it wrong and the program runs perfectly and does the wrong thing — **no error message**. The first two cannot go wrong that way.

Making a decision `IF / ELSE`

```
if (landerY > 200) {
  // runs only when true
  message = "LOW";
} else {
  // runs only when false
  message = "HIGH";
}
```

PART	ITS JOB
`( )`	holds the **question**. Whatever is inside has to come back `true` or `false` — a Boolean.
`{ }`	holds **what happens**.
`else`	holds what happens when the answer is `false`.

> **THE RULE**
> If there is an `else`, **exactly one side always runs.** Never both. Never neither.

`else` is not required

An `if` on its own is already complete. If the answer is `false`, the braces are simply skipped — nothing happens, and there is no error.

You add an `else` when the false side also has something to do.

One thing, decided two ways

```
} else {
  message = "HIGH";
}
text(message, 20, 40);
```

The last line sits **outside** the braces, so it happens either way. There is one readout — the `if` / `else` only chose what it says.

Why a decision needs a variable

```
if (landerY > 200)   // a real question
if (240 > 200)       // settled already
```

A plain number gives the same answer forever, so there is nothing to decide. A **variable** is what gives a program something to decide *about*.

What this looks like on the AP exam REFERENCE SHEET

JavaScript — what you type

```
if ( condition ) {
  // ...
} else {
  // ...
}
```

AP Exam Reference Sheet

```
IF (condition)
{
  <block of statements>
}
ELSE
{
  <block of statements>
}
```

> **IT IS NOT ANOTHER LANGUAGE**
> The exam writes programs in a notation of its own, but it is the same idea in different clothes: a question in brackets, a block in braces that runs when the answer is true. A few of the symbols differ from JavaScript's. We will go through which ones, and why, as they come up.

Three mistakes worth knowing about in advance

WRITTEN LIKE THIS	WHAT ACTUALLY HAPPENS
`if (landerY > 200);`	A semicolon straight after the `if` ends the statement early. The block below then runs **every time**, whatever the answer was. **No error message appears.**
`if (landed == "true")`	Comparing a Boolean against a *string*. Never true — and again, no error message.
`circle(200, landerY - 65, 26);` `fill(90, 220, 120);`	`fill()` is a *setting*, and p5 had already drawn the shape before the colour changed. Settings go **before** the thing they affect.

Check yourself before class

Different machine on purpose — a ground-station battery readout. The ideas are the same ones; see whether they still work when the name changes.

1. `batteryLevel` is 20. What does `batteryLevel < 20` come back as?
2. And `batteryLevel <= 20` ?
3. If the `if` side ran this time, how many times does the `else` side run?
4. Name the three types you can now recognise.
5. Why can a comparison never come back with the number 7?
6. Write an `if` that shows `"CHARGE"` when `batteryLevel` is under 15. Does it need an `else` ?

ANSWERS

1. `false` . 20 is not less than 20.
2. `true` . `<=` includes the value itself.
3. Zero. Exactly one side runs.
4. number, string, Boolean.
5. A comparison asks a question, and the only answers to a question like that are yes and no — `true` and `false` .
6. `if (batteryLevel < 15) { text("CHARGE", 20, 40); }` — no `else` needed. Nothing has to happen when it is false.

PAGE 2 — after the sandbox. Nothing here is needed for the pre-read. These are the tools behind TODO 4 and TODO 5, and the first two are on the AP exam reference sheet.

The remainder `%` · MOD ON THE EXAM

`%` gives the **remainder** after a division. It is not a percent sign.

```
12 % 5   // 2  (12 = 5+5+2)
17 % 5   // 2
10 % 5   // 0  (nothing left over)
```

> **ON THE EXAM**
> Written `a MOD b`. Same meaning. **AAP-2.C.2** — and it has the same precedence as `*` and `/`, so it is worked out before any `+` or `−` around it.

Why it is useful: it *wraps*

Divide by 30 and the remainder can only ever be 0–29. However big the number gets, the answer stays in range and starts over:

FRAMECOUNT	FRAMECOUNT % 30
28	28
29	29
30	0 ← wraps
31	1

p5's frame counter FRAMECOUNT

`frameCount` is a variable p5 gives you — you never declare it, the same way you never declare `width` or `mouseX`. It starts at 1 and goes up by one every frame. At roughly 60 frames a second it passes 3,000 in under a minute.

Once every N frames

```
if (frameCount % 60 == 0) {
  timer = timer + 1;
}
```

True on frames 60, 120, 180 … — about once a second.

Half on, half off

```
if (frameCount % 20 < 10) {
  // bright — half the cycle
} else {
  // dim — the other half
}
```

That is a blink. Smaller number, faster blink.

A different number every frame RANDOM()

```
// a decimal somewhere between −8 and 8, new every frame
random(−8, 8)
```

> **ONE NUMBER IS NOT THE SAME AS TWO**
> `random(8)` gives 0 to 8 — never negative. A flame that only ever gets longer, never shorter.

> **ON THE EXAM**
> `RANDOM(a, b)` returns a random **whole number** from `a` to `b`, including both ends. p5's version returns a decimal. Same idea, different kind of number.

An `if` inside an `if` NESTED CONDITIONALS

```
if (landerY > 200) {
  // ...
} else {
  if (frameCount % 20 < 10) {
    // ...
  }
}
```

The inner `if` is only ever reached when the outer one was **false**. It never gets asked otherwise.

> **WORTH KNOWING**
> The exam calls this a **nested conditional** and gives it a topic of its own (**3.7**). We come back to it properly later in the year — today you only need to see that it is allowed.

Asking the same question once instead of three times

> **IF THREE IFS ALL ASK THE SAME THING**
> … they can usually become one `if` with everything inside it. Same result, shorter file, and one place to fix if the question ever changes. **Watch the order**: whichever `fill()` a branch ends on is the colour the next shape comes out.

AP CSP 2026–27 · Unit 2 Lesson 1 · 3.5 Boolean Expressions (AAP-2.E) · 3.6 Conditionals (AAP-2.G, AAP-2.H) · Types (AAP-1.A.3) · NOT (AAP-2.F) · MOD (AAP-2.C.2) · Nested conditionals (3.7, AAP-2.I)